NMPC and Pure Pursuit Fallback: Models and Derivations

Oct 21, 2026 · @harsh

1. Overview

The car is driven by a nonlinear model predictive controller (NMPC). It replans every 200 ms over a 4 s horizon on a dynamic bicycle model with Fiala tires. A pure pursuit controller runs alongside at 50 Hz and takes over, blended in over 0.2 s, whenever the NMPC has no plan for the current moment.

This document derives every equation in both controllers, as implemented on the branch as of the date above. It covers the vehicle and tire model, the equations of motion, the change to road coordinates, the discretisation, the NMPC's cost and constraints, and the pure pursuit laws and fallback logic.

Where the code lives

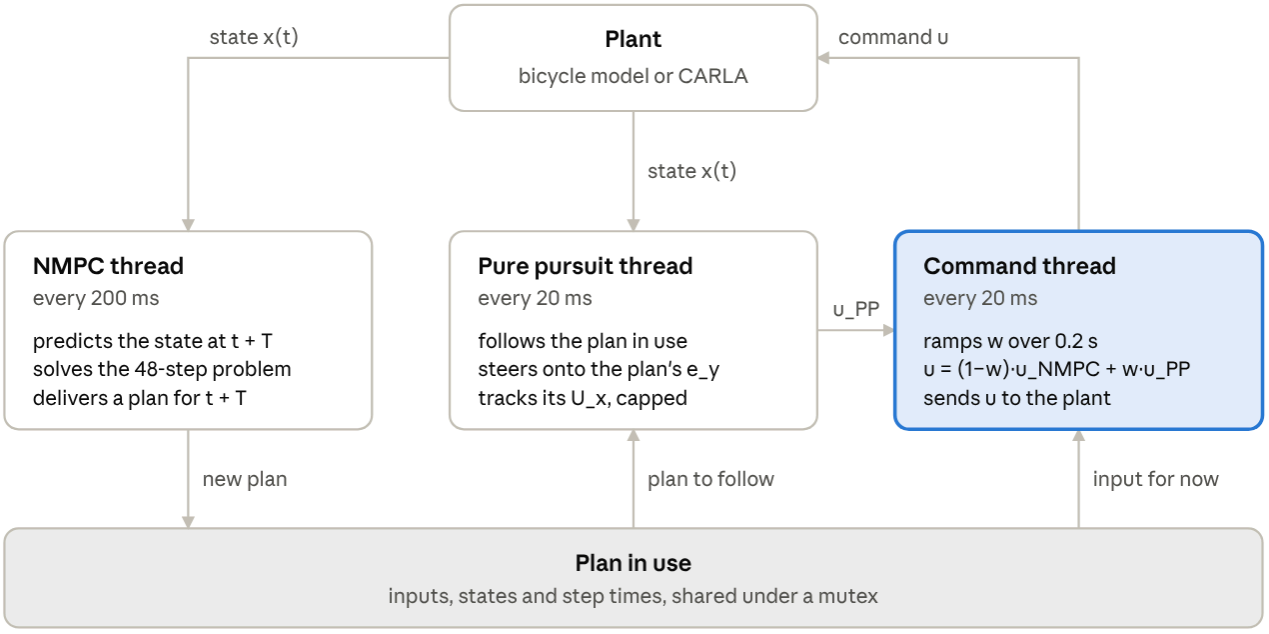
Part	File
Vehicle parameters	`track_control/include/track_control/vehicle_params.hpp`
Tire model (Fiala, friction circle)	`track_control/include/track_control/model/tire.hpp`
Equations of motion, slip angles, force limits	`model/bicycle.hpp`, `src/bicycle.cpp`
RK4 step	`model/integrate.hpp`
NMPC (problem, cost, constraints, solve)	`include/track_control/nmpc.hpp`, `src/nmpc.cpp`
Pure pursuit (steering, speed, force cap)	`include/track_control/pure_pursuit.hpp`, `src/pure_pursuit.cpp`
Fallback threads and blending	`sim/racetrack/fallback_control.hpp`, `.cpp`
Plant used for testing	`sim/plant/bicycle_plant.cpp`, `sim/plant/chrono_plant.cpp`

How the two controllers run together

Three threads share the plant (`FallbackControl`):

- **NMPC thread:** a solve every period T = 0.2 s. A solve started at plant time t plans from t + T, from the state the car is predicted to reach by then under the plan in use. Each plan is played for one period: 10 commands at 50 Hz.
- **Pure pursuit thread:** every 20 ms, from the latest measured state. It follows the line and speed of the NMPC plan in use, while that plan lasts.
- **Command thread:** every 20 ms, sends u = (1 − w)·u_NMPC + w·u_PP. The weight w ramps to 0 while the NMPC has a plan for the moment and to 1 while it has none, covering the full range in `blend` = 0.2 s.

Pure pursuit is blended in whenever the NMPC has no plan for the moment



fallback control · 3 threads, 1 shared plan

The NMPC writes a new plan every 200 ms; pure pursuit and the command thread read it every 20 ms, and the command thread alone talks to the plant.

Notation

Symbol	Meaning	Unit
s	distance along the reference path	m
e_y	lateral offset of the centre of gravity (CG) from the path, positive left	m
e_ψ	heading of the car relative to the path tangent	rad

Symbol	Meaning	Unit
U_x, U_y	velocity of the CG in the car's frame, forward and left	m/s
r	yaw rate, positive counter-clockwise (turning left)	rad/s
δ	road-wheel steering angle, positive left	rad
$\dot{\delta}$	steering rate (input)	rad/s
F_x	total longitudinal tire force (input), negative braking	N
κ(s)	path curvature, positive for a left bend	1/m
α_f, α_r	front and rear slip angles	rad
F_zf, F_zr	normal loads on the front and rear axles	N
F_yf, F_yr	lateral tire forces, in each axle's wheel frame	N

The x axis points forward and y to the left, so all angles are positive counter-clockwise seen from above. CARLA is left-handed and is converted at its boundary.

2. Vehicle model: state, inputs and parameters

Both controllers use a dynamic single-track (bicycle) model. Each axle's two wheels are lumped into one, at distance a ahead of and b behind the CG. The model has 7 states and 2 inputs (`types.hpp`).

$$x = \begin{bmatrix} X & Y & \psi & U_x & U_y & r & \delta \end{bmatrix}^\top, \qquad u = \begin{bmatrix} \dot\delta & F_x \end{bmatrix}^\top$$

The NMPC replaces the world pose (X, Y, ψ) with the road-frame pose (s, e_y, e_ψ), derived in section 5. The other four states are the same.

Why steering rate is the input. With δ as a state and $\dot{\delta}$ as the input, the steering cannot jump between steps. The rate limit then becomes a simple bound on the input, and a cost on $\dot{\delta}$ smooths the steering. The NMPC passes its longitudinal input to the solver as F_x / m, which keeps the decision variables of similar size.

Parameters

The values come from `config/vehicles/model3.json`, the CARLA Tesla Model 3 used for the CARLA and bicycle plants. They match the defaults in `vehicle_params.hpp`. The Source column is the file's own label.

Parameter	Symbol	Value	Source
Mass	m	1845 kg	CARLA
Yaw inertia	I_zz	3958 kg·m²	assumed
CG to front axle	a	1.168 m	derived
CG to rear axle	b	1.836 m	derived
CG height	h	0.45 m	assumed
Front cornering stiffness (axle)	C_αf	106 500 N/rad	derived
Rear cornering stiffness (axle)	C_αr	67 800 N/rad	derived
Tire-road friction	μ	0.9	assumed
Front share of drive force	drive_front	0.5	assumed
Front share of brake force	brake_front	0.5	derived
Peak drive force	F_drive,max	18 073 N	derived
Peak brake force	F_brake,max	7568 N	derived
Drive power	P_max	700 kW	derived
Drag area	C_dA	0.483 m²	derived
Rolling resistance	C_rr	0.012	assumed
Air density	ρ	1.225 kg/m³	constant
Steering angle limit	δ_max	0.855 rad (49°)	derived
Steering rate limit	δ̇_max	0.6 rad/s (34°/s)	design
Speed floor in slip angles	Ux_min	1 m/s	design

The body used for the road-edge and clearance constraints is 4.79 m long and 2.16 m wide, with the CG 0.45 m ahead of the body's centre (`Body` , `controller.hpp`). The road (`Road`) has 3.5 m lanes, 3.5 m from the path to the edge of the asphalt, μ = 0.9 and an 80 km/h speed limit.

Quantities derived from the parameters

Wheelbase L = a + b = 3.004 m. Taking moments about each contact patch gives the static axle loads:

$$F_{zf,0} = \frac{m g\, b}{L} = 11\,062\ \text{N}\ (61\%), \qquad F_{zr,0} = \frac{m g\, a}{L} = 7\,037\ \text{N}\ (39\%)$$

The understeer gradient follows from the steady-state cornering balance, derived in section 11:

$$K_{us} = \frac{m}{L}\left(\frac{b}{C_{\alpha f}} - \frac{a}{C_{\alpha r}}\right) \approx 7.6\times10^{-6}\ \text{rad per m/s}^2$$

So the car is almost exactly neutral-steer in the linear range: 0.004° of extra steering per g. Near the limit the Fiala model, not K_us, governs its balance.

Where the controllers' model differs from the plant

Effect	Bicycle plant	NMPC model	Pure pursuit
Cornering stiffness	follows axle load if the parameters give a table	held at the static load (`constant_stiffness`)	static load
Tire lag	slip angles relax over a relaxation length	instant	not modelled
Steering actuator	first-order lag, τ = 0.05 s	$\dot{\delta}$ applied exactly	its own τ_steer = 0.05 s, used to set the rate
Pitch and roll	quasi-static load transfer	same	same, in the force cap

Holding the stiffness constant matters for speed. When it follows the load, F_x couples into every lateral force, which roughly doubles the cost of a solve (comment on `constant_stiffness`, `nmpc.cpp`).

3. Tire model

Each axle's lateral force comes from three steps: the axle's slip angle, its normal load after load transfer, and the Fiala brush model limited by the friction left after F_x. The functions are `slip_angles` and `axle_forces` in `bicycle.hpp` and `fiala_lateral` in `tire.hpp`. An interactive version of this section is at Bicycle Model Slip Angles.

3.1 Slip angles

The car is a rigid body, so the velocity of any point p on it is the CG velocity plus ω × p. With ω = r about the vertical axis and the axles on the car's x axis at +a and −b:

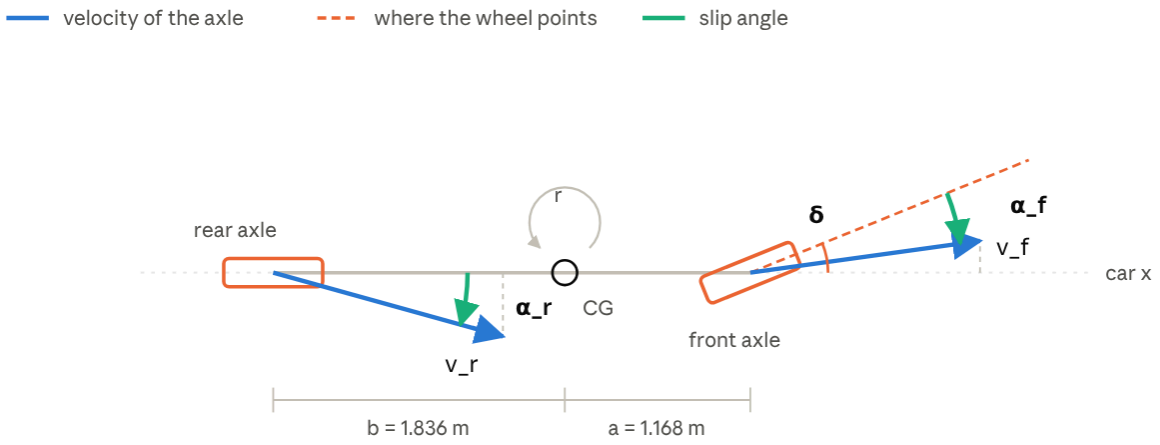
$$\mathbf{v}_f = \begin{bmatrix} U_x \\ U_y + a\, r \end{bmatrix}, \qquad \mathbf{v}_r = \begin{bmatrix} U_x \\ U_y - b\, r \end{bmatrix}$$

Yaw adds only a sideways component at each axle: forward speed is the same everywhere on the car. The slip angle is the direction of travel of the contact patch minus the direction the wheel points. The front wheel points at δ and the rear at 0:

$$\alpha_f = \arctan\frac{U_y + a\, r}{\max(U_x, U_{x,\min})} - \delta, \qquad \alpha_r = \arctan\frac{U_y - b\, r}{\max(U_x, U_{x,\min})}$$

The floor $U_{x,\min}$ = 1 m/s avoids dividing by zero at standstill. The NMPC also bounds $U_x \geq 3$ m/s at every step after the first, because the model is not meant for low speeds.

Each slip angle is the travel direction minus the wheel heading



front: $v_f = (U_x, U_y + a\cdot r)$, $\alpha_f = \operatorname{atan}((U_y + a\cdot r) / U_x) - \delta$

rear: $v_r = (U_x, U_y - b\cdot r)$, $\alpha_r = \operatorname{atan}((U_y - b\cdot r) / U_x)$

Sideways velocities and angles drawn about five times true size. Turning left: r positive, both slip angles negative.

slip angles · velocity at each axle against wheel heading

Yaw pushes the front axle's velocity left and the rear's right, so in a left turn both tires slip with negative α and push the car left.

3.2 Longitudinal force split and load transfer

F_x is split between the axles by a fixed share, one for driving and one for braking (both 0.5):

$$F_{xf} = \phi\, F_x, \quad F_{xr} = (1-\phi)\, F_x, \qquad \phi = \begin{cases} \text{drive_front} & F_x \ge 0 \\ \text{brake_front} & F_x < 0 \end{cases}$$

The load transfer is quasi-static: the pitch motion is assumed to settle instantly. Take moments about the rear contact patch. Gravity m g acts at distance b ahead of it and the front load F_zf at L. The tire forces act at the ground, h below the CG, so the inertial force −m·a_x ≈ −F_x at the CG adds a moment F_x·h. Balancing:

$$F_{zf}\, L = m\, g\, b - h\, F_x \;\Rightarrow\; F_{zf} = \frac{m\, g\, b - h\, F_x}{L}, \qquad F_{zr} = \frac{m\, g\, a + h\, F_x}{L}$$

The two loads sum to m g whatever F_x is. Braking (F_x < 0) moves load forward. At full braking (−7568 N) the rear load drops from 7037 N to 5903 N, a 16% loss of rear grip. Drag and aerodynamic lift are left out of the load transfer.

3.3 The friction circle

A tire can transmit at most μ·F_z in any direction in the road plane. If the axle already carries F_x,axle, the lateral force left is:

$$F_{y,\max} = \sqrt{(\mu F_z)^2 - F_{x,\text{axle}}^2}$$

This has a square-root singularity where F_x uses all the grip, which a gradient-based solver cannot cross. The code smooths it (`lateral_capacity`, `tire.hpp`) with F_min = 50 N:

$$F_{y,\max} = \sqrt{\tfrac12\left(\sigma + \sqrt{\sigma^2 + 4F_{\min}^4}\right)}, \qquad \sigma = (\mu F_z)^2 - F_{x,\text{axle}}^2$$

The inner term is a smooth approximation of max(σ, 0). For σ ≫ F_min² it equals σ, giving the exact circle. At σ = 0 it gives F_min², so F_y,max = 50 N rather than 0. For σ < 0 it decays smoothly towards zero. The result has a continuous derivative everywhere.

This is friction-circle derating: F_x is taken as given and lateral force gets what remains. It does not model the coupling of longitudinal and lateral slip in a combined-slip tire model.

3.4 The Fiala brush model

The physical picture. The contact patch, of half-length ℓ, is modelled as a row of elastic bristles. Each has lateral stiffness c_p per unit length. The normal pressure along the patch is parabolic:

$$q(\xi) = \frac{3F_z}{4\ell}\left(1 - \frac{\xi^2}{\ell^2}\right), \qquad -\ell \le \xi \le \ell$$

A bristle enters the patch at the leading edge $\xi = \ell$ undeflected. While it sticks to the road, travelling at slip angle α deflects it linearly with distance from the leading edge: $v(\xi) = (\ell - \xi)\tan\alpha$. Its force per unit length is $c_p \cdot v(\xi)$, but it cannot exceed the friction limit $\mu \cdot q(\xi)$ there. Near the leading edge the bristles stick. Towards the trailing edge the pressure falls to zero, so there they slide.

Cornering stiffness. If the whole patch sticks (small α), integrating $c_p(\ell - \xi)\tan\alpha$ over the patch gives $F_y = -2c_p\ell^2 \tan\alpha$. So $C_\alpha = 2c_p\ell^2$ is the slope at the origin. The minus sign means the force opposes the slip.

Partial sliding. Write $z = \tan\alpha$ and $F_{y,\max}$ for the available grip (μF_z, derated as above). Integrating the sticking region's elastic force and the sliding region's friction force, with the boundary where $c_p v = \mu q$, gives a cubic in z:

$$F_y = -C_\alpha z + \frac{C_\alpha^2}{3F_{y,\max}}\,|z|\,z - \frac{C_\alpha^3}{27F_{y,\max}^2}\,z^3, \qquad |z| < z_{sl} = \frac{3F_{y,\max}}{C_\alpha}$$

$$F_y = -F_{y,\max}\,\operatorname{sign}(z), \qquad |z| \ge z_{sl}$$

Checks at the sliding point. At $z = z_{sl}$ the three terms are $-3F$, $+3F$ and $-F$ ($F = F_{y,\max}$), so $F_y = -F_{y,\max}$. The slope there is $-C + 2C - C = 0$. The two branches therefore meet with the same value and the same slope. The $|z| \cdot z$ term (instead of z^2) keeps the curve odd, so left and right turns are symmetric.

What it means for driving. With $\mu = 0.9$ and no F_x, the front axle saturates at $\alpha = \arctan(3 \times 9956 / 106\,500) = 15.7°$ and the rear at $\arctan(3 \times 6333 / 67\,800) = 15.6°$. Braking shrinks $F_{y,\max}$ and so moves both the peak force and the sliding angle down.

3.5 Cornering stiffness against load

The parameters may give each axle a table of C_α against F_z. The plant interpolates it with a monotone cubic (Fritsch–Carlson), flat beyond its end points so the derivative is continuous. The NMPC and pure pursuit both evaluate the stiffness once at the static load and hold it there.

4. Equations of motion

The motion follows from Newton's second law for translation and Euler's for yaw, written in the car's own frame (`dynamics`, `bicycle.hpp`). An interactive version is at Bicycle Model in Road Coordinates.

4.1 Acceleration in a rotating frame

Let i and j be unit vectors along the car's x and y axes. They turn with the car at r, so their time derivatives are di/dt = r·j and dj/dt = −r·i. Differentiating the CG velocity v = U_x i + U_y j:

```
\mathbf a = \dot U_x \mathbf i + U_x r\, \mathbf j + \dot U_y \mathbf j - U_y r\, \mathbf i = (\dot U_x - r U_y)\,\mathbf i + (\dot U_y + r U_x)\,\mathbf j
```

Setting m·a equal to the summed forces and solving for the rates:

```
\dot U_x = \frac{\sum F_x}{m} + r U_y, \qquad \dot U_y = \frac{\sum F_y}{m} - r U_x
```

The terms r·U_y and −r·U_x carry no force. They appear because the velocity is measured in a frame that turns. In steady cornering, U̇_y = 0 and so ΣF_y = m·r·U_x = m·U_x²/R: the tires supply exactly the centripetal force.

4.2 Forces in the car's frame

The front axle's forces act in the wheel frame, turned by δ. Rotated into the car's frame they are:

```
\begin{bmatrix} F_{x,\text{front}} \\ F_{y,\text{front}} \end{bmatrix} = \begin{bmatrix} \cos\delta & -\sin\delta \\ \sin\delta & \cos\delta \end{bmatrix} \begin{bmatrix} F_{xf} \\ F_{yf} \end{bmatrix}
```

The rear wheels do not steer, so their forces are already in the car's frame. Drag acts backwards at the CG: aerodynamic drag plus rolling resistance, the latter taken as a constant force.

```
F_{\text{drag}} = \tfrac12 \rho\, C_dA\, U_x^2 + C_{rr}\, m g
```

At 20 m/s this is 118 N of air drag plus 217 N of rolling resistance.

4.3 Yaw

Only lateral forces in the car's frame produce a moment about the CG, since both axles lie on the car's x axis. The front lateral force has arm a and the rear arm b, on the other side:

```
I_{zz}\, \dot r = a\, (F_{yf}\cos\delta + F_{xf}\sin\delta) - b\, F_{yr}
```

Lumping each axle's two wheels onto the centreline removes the track width. So a left–right difference in F_x (for example torque vectoring) cannot yaw the car in this model.

4.4 The complete model

```
\begin{aligned}
\dot U_x &= \frac{F_{xf}\cos\delta - F_{yf}\sin\delta + F_{xr} -
F_{\text{drag}}}{m} + r U_y \\
\dot U_y &= \frac{F_{yf}\cos\delta + F_{xf}\sin\delta + F_{yr}}{m} - r U_x \\
\dot r &= \frac{a\,(F_{yf}\cos\delta + F_{xf}\sin\delta) - b\, F_{yr}}
{I_{zz}} \\
\dot X &= U_x\cos\psi - U_y\sin\psi, \qquad \dot Y = U_x\sin\psi +
U_y\cos\psi, \qquad \dot\psi = r \\
\dot\delta &= u_{\dot\delta}
\end{aligned}
```

The last line integrates the steering-rate input. The X and Y rates rotate the body-frame velocity into the world frame by ψ.

The equations are written once as a C++ template on the scalar type. The simulator evaluates them in `double`; the NMPC evaluates the same code in CasADi's symbolic `SX`, from which it takes exact derivatives for the solver. The controller and the plant therefore cannot drift apart in their algebra.

4.5 Longitudinal force limits

The range of F_x the car can deliver in a straight line (`fx_limits`, `bicycle.cpp`) bounds both controllers' F_x. Each axle's share of F_x must stay within its friction times its load, with the load transfer included. For braking at the front, φ_b·|F_x| ⩽ μ·(m g b + h |F_x|)/L, so:

```
|F_x| \le \frac{\mu m g\, b}{\phi_b L - \mu h}
```

At the rear, (1 − φ_b)·|F_x| ⩽ μ·(m g a − h |F_x|)/L, so |F_x| ⩽ μ m g a / ((1 − φ_b) L + μ h). The same is done for driving with the signs of the load transfer reversed. Each limit is also capped by the hardware and, for driving, by power:

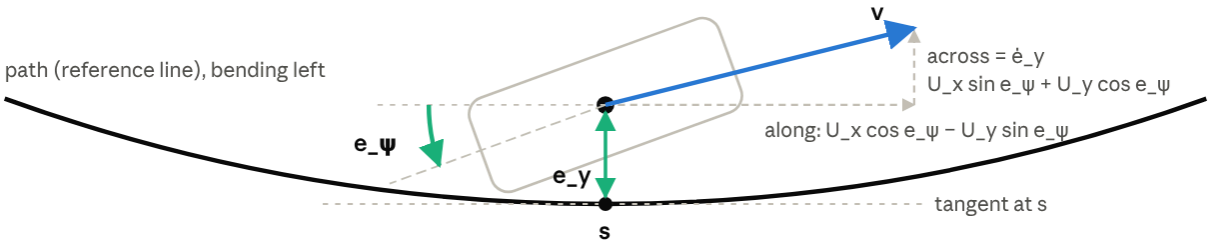
```
F_{x,\min} = -\min\left(F_{\text{front}}, F_{\text{rear}},
F_{\text{brake,max}}\right), \qquad F_{x,\max} = \min\left(F_{\text{front}},
F_{\text{rear}}, F_{\text{drive,max}}, \frac{P_{\max}}{\max(U_x,
U_{x,\min})}\right)
```

With the fixed 50/50 split one axle reaches μ·F_z first, so the grip limits sit below μ m g. For braking the rear limits first, at 0.9 × 18 099 × 1.168 / (0.5 × 3.004 + 0.9 × 0.45) = 9977 N. That is above the brakes' 7568 N, so in a straight line the brakes are the limit.

5. Road coordinates

The NMPC carries the car's pose relative to the path: s along it, e_y across it and e_ψ against its tangent. In these coordinates a corner enters only through κ(s), and the road edges become bounds on e_y. The equations below are the lambda `f` in the NMPC constructor (`nmpc.cpp`).

e_y and e_ψ place the car against the path's tangent at s



ṡ = (U_x cos e_ψ − U_y sin e_ψ) / (1 − κ·e_y)
ė_y = U_x sin e_ψ + U_y cos e_ψ
ė_ψ = r − κ·ṡ

Not to scale: the offset, the heading error and the bend are drawn larger than true size.

road frame · s, e_y, e_ψ and the velocity split

The velocity splits into a part along the path, which advances s, and a part across it, which is ė_y; e_ψ is measured between the car's axis and the tangent at s.

5.1 Frenet frame of the path

Let P(s) be the path point at arc length s, with unit tangent t(s) and unit left normal n(s). The tangent's angle is θ_p(s), and the curvature is defined by dθ_p/ds = κ. Differentiating the two unit vectors along the path (the Frenet–Serret formulas in the plane):

```
\frac{d\mathbf t}{ds} = \kappa\, \mathbf n, \qquad \frac{d\mathbf n}{ds} = -\kappa\, \mathbf t
```

5.2 Position and velocity

The CG is at the foot point plus the offset along the normal, so p = P(s) + e_y·n(s). Differentiating with respect to time, using dP/ds = t and the chain rule:

$$\dot{\mathbf p} = \dot s\, \mathbf t + \dot e_y\, \mathbf n + e_y\, \dot s\, (-\kappa\, \mathbf t) = \dot s\,(1 - \kappa e_y)\, \mathbf t + \dot e_y\, \mathbf n$$

The same velocity is (U_x, U_y) in the car's frame, which is turned by e_ψ = ψ − θ_p from the path frame. Projecting onto t and n:

$$\dot{\mathbf p}\cdot\mathbf t = U_x\cos e_\psi - U_y\sin e_\psi, \qquad \dot{\mathbf p}\cdot\mathbf n = U_x\sin e_\psi + U_y\cos e_\psi$$

Equating the two expressions component by component:

$$\dot s = \frac{U_x\cos e_\psi - U_y\sin e_\psi}{1 - \kappa e_y}, \qquad \dot e_y = U_x\sin e_\psi + U_y\cos e_\psi$$

Why the divisor. s is measured on the path, not where the car is. On a left bend (κ > 0) a car left of the path (e_y > 0) runs on a circle of radius 1/κ − e_y. The same angle around the centre covers 1/κ of path for every 1/κ − e_y the car travels, so s advances faster than the car by 1/(1 − κ e_y). The model is singular at e_y = 1/κ, the centre of curvature. The road-edge bound, about 2.2 m from the path, keeps it far from there, since the tightest corner has a radius well above that.

5.3 Heading

e_ψ = ψ − θ_p(s), and ψ̇ = r. The path's tangent turns at dθ_p/dt = κ·ṡ:

$$\dot e_\psi = r - \kappa\, \dot s$$

In steady cornering on the path, r = κ·ṡ and e_ψ stays constant. A nonzero steady e_ψ equals minus the sideslip angle β = arctan(U_y/U_x): the body points slightly inside or outside the path while the velocity follows it.

5.4 The NMPC's state equation

$$f(z,\, u;\, \kappa) = \begin{bmatrix} \dot s & \dot e_y & \dot e_\psi & \dot U_x & \dot U_y & \dot r & \dot\delta \end{bmatrix}^\top, \qquad z = \begin{bmatrix} s & e_y & e_\psi & U_x & U_y & r & \delta \end{bmatrix}^\top$$

The code reuses `dynamics` from section 4 with ψ set to e_ψ. Its Ẋ and Ẏ then come out as the velocity along and across the path, and the lambda turns them into ṡ, ė_y and ė_ψ as above. The last four rows are the equations of motion unchanged.

5.5 Curvature along the horizon

The curvature is a parameter per step, not a function of the decision variable s. That keeps the problem smooth and its derivatives cheap. Each solve takes the previous plan, shifted to this solve's time, as its estimate s_plan[k] of where step k will be. Step k's curvature is the mean of five samples of the known curvature, evenly spaced from s_plan[k] to s_plan[k+1]:

```
\bar\kappa_k = \frac15 \sum_{i=0}^{4} \kappa_{\text{known}}\left(s_k + \tfrac{i}{4}(s_{k+1} - s_k)\right), \qquad \kappa_{\text{known}}(s) = \kappa\big(\min(s,\ s_0 + \text{road\_range})\big)
```

Beyond the known road (40 m) the last known curvature is held. The first solve has no previous plan, so it assumes the car carries on at its measured speed: s_plan[k] = s_0 + U_x·t_k.

5.6 From the measured state

`model_state` projects the measured world pose onto the path (`path_state`) to get (s, e_y, e_ψ), and copies U_x, U_y, r and δ unchanged. s is unwrapped within one solve, so the plan never jumps at the start/finish line.

6. Discretisation

The continuous model is turned into 48 discrete steps over 4 s. Each step is one RK4 step with the input held constant, and every step's state is a decision variable tied to the next by an equality constraint (multiple shooting).

6.1 The step grid

`make_steps` builds `h_fine` steps for the first `fine` seconds and steps of h after them, out to the horizon. With `nmpc_fallback` the bridge sets h_fine to one command period and fine to one solve period (`carla_bridge/run.py`):

Steps	Length	Covers	Why
k = 0–9	20 ms	0–0.2 s	the part of the plan that is played, one step per 50 Hz command
k = 10–47	100 ms	0.2–4.0 s	the part that only shapes the first 0.2 s

So N = round(0.2 / 0.02) + round(3.8 / 0.1) = 10 + 38 = 48. A uniform 20 ms grid over 4 s would need 200 steps; this grid gets the same resolution where commands are sent for a quarter of the cost. The step start times are t_k = h_0 + … + h_(k−1), and `step_at(t)` finds the step a time falls in.

6.2 One RK4 step

The input u_k is held over the step (zero-order hold), and the curvature is that step's mean $\bar{\kappa}_k$. The classical fourth-order Runge–Kutta step (`rk4_step` , `integrate.hpp`) is:

```
\begin{aligned}
k_1 &= f(z_k,\ u_k), & k_2 &= f\!\left(z_k + \tfrac{h_k}{2} k_1,\ u_k\right),
\\
k_3 &= f\!\left(z_k + \tfrac{h_k}{2} k_2,\ u_k\right), & k_4 &= f(z_k + h_k
k_3,\ u_k), \\
F(z_k, u_k) &= z_k + \tfrac{h_k}{6}\,(k_1 + 2k_2 + 2k_3 + k_4)
\end{aligned}
```

RK4 matches the Taylor expansion of the exact solution to fourth order in h. Its error is $O(h^5)$ per step and $O(h^4)$ over a fixed time span. Halving the 0.1 s step would cut the coarse grid's error about 16-fold. One step per stage keeps the problem small; the cost is 4 model evaluations per stage, 192 per evaluation of all the dynamics.

6.3 Multiple shooting

Every stage's state is a decision variable, not something computed by running the model forward from x_0. The variables are laid out stage by stage:

```
w = \begin{bmatrix} z_0 & u_0 & \varepsilon_0 & z_1 & u_1 & \varepsilon_1 &
\cdots & z_{N-1} & u_{N-1} & \varepsilon_{N-1} & z_N \end{bmatrix}
```

That is 7 states, 2 inputs and 4 slacks ε per stage, plus the final state: 48 × 13 + 7 = 631 variables. The dynamics become equality constraints, and the first state is pinned to the measurement:

```
z_{k+1} - F(z_k, u_k) = 0, \quad k = 0, \dots, N-1, \qquad z_0 -
z_{\text{init}} = 0
```

This has three advantages over single shooting (where only the inputs are variables):

- **Conditioning.** An early input does not have to steer a long nonlinear chain through 48 steps. The solver can start with gaps between stages and close them as it converges.
- **A good start.** The previous plan, shifted in time, gives a starting guess for every state, not just the inputs (section 9).

- **Structure.** Each constraint and cost term depends on one stage's variables only. FATROP exploits this block structure with a Riccati recursion over the stages (section 9).

6.4 Where the costs and constraints are evaluated

Each stage's cost and constraint terms are evaluated at the state F(z_k, u_k), the end of step k as that stage predicts it. Once the gaps are closed this equals z_(k+1). Writing it this way keeps every term inside one stage, as FATROP's structure needs. Each stage's cost is also weighted by its step's length relative to h = 0.1 s, so the cost is a sum over time whatever the steps:

```
J = \sum_{k=0}^{N-1} \frac{h_k}{h} \sum_i w_i\, \ell_i(\text{stage}_k)
```

Without this weighting, the ten 20 ms stages would count as much as the ten 0.1 s stages that cover five times as long.

6.5 Bounds on the variables

Variable	Bound	Where
δ (k ⩾ 1)	±0.855 rad	steering limit
U_x (k ⩾ 1)	⩾ 3 m/s	the tire model is not meant for lower speeds
δ̇	±0.6 rad/s	steering-rate limit
F_x / m	[F_x,min, F_x,max] / m from `fx_limits` at the measured U_x	set again every solve
slacks ε	⩾ 0	section 8

Stage 0 is left unbounded because it is fixed to the measured state anyway.

7. NMPC cost function

The cost has six named terms, each with a weight (`cost_terms` , `nmpc.cpp`). Two drive the car (speed and lane), two keep it settled (lateral motion and the inputs), and one makes the soft constraints nearly hard. No speed is prescribed per corner. The plan brakes for a corner only because the tire model, the road edges and the sideslip limit leave it no other way through.

```
J = \sum_{k=0}^{N-1} \frac{h_k}{h}\Big[ w_v (U_x - v_{\text{ref}})^2 + w_e
(e_y - e_{\text{ref}})^2 + w_{\text{lat}}\big(\dot e_y^2 + (r - \kappa
U_x)^2\big) + w_a \Big(\frac{F_x}{m}\Big)^2 + w_{\dot\delta}\, \dot\delta^2 +
w_\varepsilon \sum_j (\varepsilon_j + \varepsilon_j^2) \Big]
```

The states are those at the end of step k (section 6.4), and the inputs are step k′s.

Term	Expression	Weight	What 1 unit of cost buys
speed	$(U_x - v_{ref})^2$	1.0	1 m/s below the speed asked for
lane	$(e_y - e_{ref})^2$	0.2	2.2 m off the home lane
lat	$\dot{e}_y^2 + (r - \kappa U_x)^2$	1.0	1 m/s of drift across the path, or 1 rad/s of yaw rate away from the path′s
accel	$(F_x/m)^2$	1.0	1 m/s² of acceleration or braking
rate	$\dot{\delta}^2$	100	0.1 rad/s (5.7°/s) of steering rate
slack	$\Sigma(\varepsilon + \varepsilon^2)$	1000	1 mm of constraint violation

7.1 Speed

v_ref is the speed limit, min(speed_cap, v_max) = 22.2 m/s (80 km/h) by default. Asking for the limit everywhere means the plan drives as fast as the constraints allow. Speed is softly asked for here; the separate hard-ish speed limit (section 8.4) handles what the car must not exceed.

7.2 Lane

e_ref is the home lane, `lane_offset` = −1.75 m, the centre of the right lane (the circuit is driven clockwise). The weight is low (0.2) so the plan leaves the lane freely to pass a car or to cut a corner, but returns when there is no reason not to.

7.3 Lateral motion

This term damps motion relative to the path. The first part is the drift across the path, $\dot{e}_y = U_x \sin e_\psi + U_y \cos e_\psi$. The second is the difference between the yaw rate and $\kappa \cdot U_x$, the yaw rate that following the path at the current speed needs.

Without this term the lane cost alone is a spring with no damper. The plan would swing towards the lane centre and overshoot. Penalising lateral velocity and yaw-rate error is the damper, and makes the approach to the line well behaved.

7.4 Inputs

The acceleration cost keeps the plan from using full braking or driving without need, and makes the speed term's trade-off explicit. Closing a 4 m/s speed gap by braking at 4 m/s² for one stage costs as much as carrying that gap for one stage. The steering-rate cost, with weight 100, is what makes the NMPC's steering smoother than pure pursuit's, which runs at its rate limit much of the time.

7.5 Slack

Each of the four constraint groups is softened by one slack per stage (section 8). The penalty $w_\varepsilon(\varepsilon + \varepsilon^2)$ has both a linear and a quadratic part.

The linear part makes the penalty exact. If a hard-constrained solution exists and w_ε is larger than the constraint's Lagrange multiplier, the soft problem's optimum has $\varepsilon = 0$ exactly. A quadratic penalty alone always lets a small violation through, since its slope at zero is zero. The quadratic part keeps the problem strongly convex in ε, so a large violation, if one is unavoidable, is spread over stages rather than piled onto one.

7.6 Changing the weights at run time

Every weight is a parameter of the problem, not a constant compiled into it. A supervisor (for example a vision-language model) can change them between solves with `set_directive`, without rebuilding the problem. Each weight is clamped to 0.1–10× its default. The slack weight may only go up, so constraints can never be made softer. A directive lapses after its time to live (2 s by default, at most 60 s).

8. NMPC constraints

Besides the dynamics and the variable bounds, there are four groups of inequality constraints (`constraint_groups`, `nmpc.cpp`). Each group shares one slack per stage: every expression g in group j must satisfy $g \leqslant \varepsilon_{j,k}$ with $\varepsilon_{j,k} \geqslant 0$, and the slack is penalised as in section 7.5. A plan that cannot keep a constraint therefore still exists, and the slack says by how much it is broken.

Group	Expressions per stage	Limit (default)
obstacle	1 per car slot (2) and per zone slot	outside an ellipse around each
edge	4: both body ends, both edges	body within 2.22 m of the path
sideslip	2	$\|\beta\| \leqslant 7°$
speed	1	$U_x \leqslant v_{lim,k}$

8.1 Road edges

The body must stay on the asphalt with a margin. The furthest the body's centreline may be from the path is:

$$e_{\max} = \text{half_width} - \tfrac12\,\text{width} - \text{edge_margin} = 3.5 - 1.08 - 0.2 = 2.22\ \text{m}$$

The body's ends stick out further than the CG when the car is at an angle to the path. Each end is at a distance ℓ along the car's axis from the CG, so its offset is $e_y + \ell \cdot \sin e_\psi$. The CG is 0.45 m ahead of the body's centre, so the arms are:

$$\ell_{\text{front}} = \tfrac12 (4.79) - 0.45 = 1.945\ \text{m}, \qquad \ell_{\text{rear}} = -\left(\tfrac12 (4.79) + 0.45\right) = -2.845\ \text{m}$$

$$e_y + \ell\sin e_\psi - e_{\max} \le \varepsilon_{\text{edge}}, \qquad -e_{\max} - (e_y + \ell\sin e_\psi) \le \varepsilon_{\text{edge}}, \qquad \ell \in \{\ell_{\text{front}}, \ell_{\text{rear}}\}$$

This treats the end's offset as measured in the path frame at s, which holds when the curvature changes little over a car length.

8.2 Other cars and keep-out zones

Prediction. Each detected car is assumed to hold its speed and its lateral offset. It moves along the line at that offset, so its arc length on the reference path at each step is found through `offset_to_s`. The two nearest cars, by distance along the path, are planned with. Their s values are unwrapped across the lap and placed around the ego's s_0.

The ellipse. Each car is kept out by an ellipse centred on it, in (s, e_y), measured from the ego's body centre (CG − 0.45 m):

$$1 - \left(\frac{s - 0.45 - s_{\text{car},k}}{A}\right)^2 - \left(\frac{e_y - e_{\text{car}}}{B}\right)^2 \le \varepsilon_{\text{obs}}$$

Choosing the half-axes. Two bodies side by side with the margins need $|\Delta e_y| \geqslant l_y$ = width + margin_lat = 2.76 m. Nose to tail they need $|\Delta s| \geqslant l_x$ = length + margin_long = 5.79 m. The ellipse is made a lane wide, less 0.4 m, so that passing in the next lane stays outside it: B = 3.5 − 0.4 = 3.1 m. Requiring the corner (l_x, l_y) of the combined rectangle to lie on the ellipse gives A:

$$\left(\frac{l_x}{A}\right)^2 + \left(\frac{l_y}{B}\right)^2 = 1 \;\Rightarrow\; A = \frac{l_x}{\sqrt{1 - (l_y/B)^2}} = \frac{5.79}{\sqrt{1 - (2.76/3.1)^2}} = 12.7\ \text{m}$$

A keep-out zone from a directive, of length ℓ_z and width w_z, is covered by the smallest-area ellipse through the corners of the zone grown by the ego's body and margins. For a rectangle of half-sides (p, q) that ellipse has half-axes ($\sqrt{2}\cdot p$, $\sqrt{2}\cdot q$):

$$A_z = \sqrt2\left(\tfrac12(\ell_z + \text{length}) + \text{margin_long}\right), \qquad B_z = \sqrt2\left(\tfrac12(w_z + \text{width}) + \text{margin_lat}\right)$$

Unused slots. The problem always has the same number of slots. An unused slot carries the flag on = 0 and the expression $\text{on}\cdot(\ldots) + (\text{on} - 1) = -1$, a constant the solver never sees as active. Parking an unused obstacle far away instead gives a constraint near -10^8, which scales the interior-point method's slacks badly and costs about half again as many iterations.

8.3 Sideslip

The sideslip angle $\beta = \arctan(U_y/U_x)$ is limited to 7°. Written as linear inequalities in the states:

$$U_y - \tan\beta_{\max}\, U_x \le \varepsilon_{\text{slip}}, \qquad -U_y - \tan\beta_{\max}\, U_x \le \varepsilon_{\text{slip}}$$

This keeps the plan inside the range where the model is accurate and away from the large rear slip angles where the car would spin. A directive may tighten it to as little as 2°.

8.4 Speed limit and the braking envelope

The road is known only `road_range` = 40 m ahead. Beyond that the car must be able to stop for whatever comes into view. Here that is taken to be the tightest corner on the circuit, a property of the road known in advance. Its speed at the friction limit is:

$$v_{\text{unseen}} = \sqrt{\frac{\mu g}{\kappa_{\max}}} \approx 11.5\ \text{m/s}$$

The plan brakes at a constant deceleration a_brake = min(F_brake,max, μ m g)/m = 4.10 m/s². From v² = v_end² + 2·a·d, the fastest speed d metres before the end of the known road from which it can still slow to v_unseen is √(v_unseen² + 2·a_brake·d). The limit at step k is therefore:

```
v_{\lim,k} = \min\left(\text{speed\_cap},\ v_{\max},\
\sqrt{v_{\text{unseen}}^2 + 2\, a_{\text{brake}} \max\!\big(0,\ s_0 +
\text{road\_range} - s_{\text{plan},k+1}\big)}\right)
```

```
U_{x,k+1} - v_{\lim,k} \le \varepsilon_{\text{speed}}
```

At the start of the plan d = 40 m, which allows 21.4 m/s. So on a straight the limit, not the 22.2 m/s speed cap, sets the car's speed.

Why an envelope and not a step. The earlier version limited the speed to v_unseen only beyond the known road, a step at s_0 + 40 m. With a 2 s horizon at 20 m/s the plan's end sat almost exactly on that step. Whether its last steps were limited flipped from one solve to the next, and the plan alternated between accelerating and braking: F_x swung between −3.5 and +4.5 kN. The envelope is continuous in s, so a small change in where the plan ends changes the limit only a little. Offline, on the straights, it cut the F_x spread from 3988 N to 676 N with the 2 s horizon (`docs/todo.md`).

The limit uses s_plan, the previous plan's positions, not the decision variable s. It is a parameter per step, which keeps the constraint linear in U_x.

8.5 What is not a constraint

There is no separate friction-circle constraint. Every lateral force comes out of the Fiala model with the derated F_y,max, so the plan cannot ask more of a tire than it can give. A plan that needs more grip than there is simply does not follow the line, and runs into the edge or sideslip constraints instead.

9. Solving the problem

The problem is built once, symbolically, in the constructor. Each solve only sets its parameters and starting point and calls the solver (`NMPC::compute`). With the 4 s horizon on the bicycle stand-in, a solve takes 74 ms at the median and 94 ms at the 95th percentile, and all 368 solves of a lap finished inside the 200 ms period (`docs/todo.md`).

9.1 Parameters set at every solve

Parameter	Size	Value
z_init	7	the measured state in road coordinates
$\bar{\kappa}_k$	N	mean known curvature over each step (section 5.5)
v_lim,k	N	braking envelope (section 8.4)
v_ref,k, e_ref,k	N each	speed cap and lane offset
weights	6	defaults, or a directive's
car s per step, car e_y, ellipse half-axes	2N + 2 + 2	predicted cars (section 8.2)
zone s per step, e_y, half-axes, on flags	per zone	directive zones
e_max, tan β_max	2	edge and sideslip limits

The F_x bounds are recomputed from `fx_limits` at the measured speed and set as variable bounds, since the power limit depends on U_x.

9.2 Warm start: shifting the previous solution

The solution of one solve is a good guess for the next, moved on in time. If the previous plan was made at time t_prev and this solve is for t, each stage k takes the old solution at time t_k + (t − t_prev), interpolated linearly between its stages:

```
w_k^{\text{start}} = (1 - \lambda)\, w_j^{\text{old}} + \lambda\,
w_{j+1}^{\text{old}}, \qquad j + \lambda = \text{index of } t_k + (t -
t_{\text{prev}}) \text{ in the old grid}
```

Stages that fall beyond the old horizon repeat its last input and slacks, and its last state coasts on at constant speed in s. The s values are then shifted so that stage 0 sits at the measured s_0, and stage 0 is overwritten with the measured state. The gap between the shifted guess and the measurement, |x_0 − x_init|, is logged as a check on how good the warm start was.

The dual variables (multipliers) are shifted the same way but are not passed by default (`warm_duals` = false). IPOPT takes about 20% fewer iterations with them. FATROP, however, can get stuck with them for seconds in a plan that leaves the road (up to 183 iterations), against at most 43 from the primal start alone.

The first solve has nothing to shift. It starts from the measured state carried on at constant speed, with e_ψ, U_y, r and δ set to zero after stage 0.

9.3 The solver

The default solver is FATROP, an interior-point method like IPOPT. The difference is how each Newton step's linear system is solved.

In an interior-point method each iteration solves the KKT system of the barrier problem. For an optimal-control problem laid out stage by stage, that system is block-banded: stage k's variables couple only to stages $k-1$ and $k+1$, through the dynamics. FATROP solves it with a Riccati recursion, one backward and one forward sweep over the 48 stages, at a cost of $O(N \cdot n^3)$ with $n \approx 13$ the stage size. IPOPT treats the same system as a general sparse matrix and factorizes it with a general-purpose method.

FATROP needs to know the stage structure. CasADi finds it from the order of the variables and constraints (`structure_detection` = auto), and the code passes which constraint rows are equalities, since FATROP does not infer this from the bounds. Both solvers run to a tolerance of 10^{-3} with at most 200 iterations. IPOPT can also be given a wall-time limit; FATROP has none.

9.4 After the solve

A solve that does not converge is counted in `failures`. Running alone, the NMPC still applies its result. In the fallback setup a non-converged plan is not used: it is reported as a failure and pure pursuit takes over (section 13). Either way the solution is stored as the next warm start, and the plan is extracted:

- `plan`: the 49 predicted states in road coordinates;
- `plan_inputs`: the 48 inputs $[\dot{\delta}, F_x]$, each to be held for h_k starting at $t + t_k$.

9.5 Playing the plan

In the fallback setup a solve takes up to a period, so planning from the current state would be stale by the time the plan arrives. A solve started at plant time t therefore plans from $t + T$. The starting state is the current state predicted forward by T with the bicycle model and the inputs the plan in use will send meanwhile.

The plan's first 0.2 s (its ten 20 ms steps) is played as ten 50 Hz commands, and then the next plan takes over. A solve that runs past its period is not dropped. Its plan takes over when it arrives, from the input it planned for that moment, and the next solve starts at once. Meanwhile pure pursuit is blended in (section 13).

10. Pure pursuit: the geometric law

Pure pursuit steers the rear axle along a circular arc to a target point a lookahead distance ahead on the line. In the current code this law is the alternative (`feedforward_steering` = false). The default is the feedforward–feedback law of section 11, but both share the same speed control and target line.

10.1 The target line

The line followed is the path offset by e_target. Normally that is the home lane, −1.75 m. When a car is in the way on the home line, the target slides to `pass_offset` = +1.75 m at `lateral_rate` = 1.2 m/s, and back once the way is clear.

A car is in the way when it is within half a lane of the home line, and between 15 m behind and max(30 m, reach) ahead, where reach = body length + 3.5 s × closing speed. This leaves 3.5 s to change lane before reaching a slower car. When following an NMPC plan, the target line is the plan's e_y instead (section 13).

The curvature of a line offset by e from a path of curvature κ follows from the radius of the offset circle, 1/κ − e:

```
\kappa_{\text{line}} = \frac{1}{1/\kappa - e} = \frac{\kappa}{1 - \kappa e}
```

10.2 The lookahead point

The lookahead distance grows with speed, so the car looks further ahead when it covers ground faster:

```
L_d = \min\big(\max(k_{\text{la}}\, U_x,\ L_{d,\min}),\
\text{road\_range}\big), \qquad k_{\text{la}} = 0.6\ \text{s}, \quad
L_{d,\min} = 5\ \text{m}
```

The target point is on the target line at L_d further along the path than the rear axle. At 20 m/s that is 12 m.

10.3 Deriving the steering angle

Let the rear axle be at R, the target at T, and α the angle between the car's heading and the line R→T. Let ℓ = |RT| be the chord. In the kinematic bicycle model (no slip) the rear axle moves on a circle tangent to the car's heading. The circle through R, tangent to the heading, that also passes through T has its centre on the line through R perpendicular to the heading.

The chord RT subtends the angle 2α at the centre, since the angle between a tangent and a chord equals half the arc's central angle. So the chord length is $\ell = 2R_c \sin\alpha$, and the circle's radius is:

```
R_c = \frac{\ell}{2\sin\alpha}, \qquad \kappa_c = \frac{2\sin\alpha}{\ell}
```

In the kinematic bicycle model the steering angle that makes the rear axle follow radius R_c is $\tan\delta = L / R_c$. So:

```
\delta_{\text{cmd}} = \arctan\frac{2 L \sin\alpha}{\ell}
```

This is what `PurePursuit::compute` computes, with ℓ the actual distance to the target (close to L_d).

10.4 Why it was replaced

Pure pursuit is purely geometric: it assumes the tires do not slip. Near the grip limit the car's sideslip and the tires' slip angles are large, so the arc it steers for is not the arc the car follows. It also has no feedforward of the corner ahead; it reacts only once the target point is in the corner. In the corner-outage scenarios (section 14), pure pursuit as it was (this law, its own lane and speed profile, no force cap) left the road or spun in 103 of 120 right-turn outages.

10.5 From angle to rate

Both laws produce a steering angle, but the plant takes a steering rate. The rate closes the angle error with a time constant $\tau_{steer} = 0.05$ s, within the rate limit:

```
\dot\delta = \operatorname{clamp}\!\left(\frac{\operatorname{clamp}
(\delta_{\text{cmd}}, \pm\delta_{\max}) - \delta}{\tau_{\text{steer}}},\
\pm\dot\delta_{\max}\right)
```

With $\tau = 0.05$ s any error above $0.6 \times 0.05 = 0.03$ rad (1.7°) saturates the 0.6 rad/s rate limit. That is why pure pursuit runs at the rate limit much of the time.

11. Feedforward–feedback steering

The default steering law (`feedforward_steering` = true) follows Kapania and Gerdes (2015), *Design of a feedback-feedforward steering controller for accurate path tracking and stability at the limits of handling*. It adds a feedforward of the steady-state steering angle for the line's curvature to a lookahead feedback on the lateral and heading errors:

$$\delta_{\text{cmd}} = \underbrace{L\,\kappa_{\text{line}} + K_{us}\, U_x^2\, \kappa_{\text{line}}}_{\text{feedforward}} \; \underbrace{-\, k_p\big(e + x_{la}\,(e_\psi + \beta_{ss})\big)}_{\text{lookahead feedback}}$$

with e = e_y − e_target the CG's offset from the target line, k_p = 0.05 rad/m and x_la = 12 m.

11.1 The feedforward: steady-state steering

In steady cornering on a circle of curvature κ at speed U_x, the yaw rate is r = U_xκ and the lateral acceleration is a_y = U_x²κ.

Slip angles from the forces. The axle forces needed are found from section 4 with $\dot{U}_y = \dot{r} = 0$. The moment balance a·F_yf = b·F_yr and the force balance F_yf + F_yr = m·a_y give F_yf = m a_y b/L and F_yr = m a_y a/L. In the linear tire range F_y = −C_α·α, so:

$$\alpha_f = -\frac{m\, a_y\, b}{L\, C_{\alpha f}}, \qquad \alpha_r = -\frac{m\, a_y\, a}{L\, C_{\alpha r}}$$

Slip angles from the kinematics. For small angles, section 3.1 gives α_f = β + a r/U_x − δ and α_r = β − b r/U_x, with β = U_y/U_x. Subtracting removes β: α_f − α_r = L r/U_x − δ = Lκ − δ. Solving for δ and substituting the force expressions:

$$\delta_{ss} = L\kappa - \alpha_f + \alpha_r = L\kappa + \frac{m}{L}\left(\frac{b}{C_{\alpha f}} - \frac{a}{C_{\alpha r}}\right) U_x^2 \kappa = L\kappa + K_{us}\, U_x^2\, \kappa$$

The first term is the kinematic (Ackermann) angle and the second is the understeer correction. As section 2 showed, K_us is nearly zero for this car, so the feedforward is almost purely geometric. The stiffnesses are taken at the static loads.

Preview. The steering is rate-limited, so steering for the curvature where the car is now turns the wheel too late for a corner ahead. The code uses the mean curvature over the next 0.5 s of travel instead, five samples from s to s + 0.5·U_x. That curvature is converted to the target line's, κ/(1 − κ·e_target).

11.2 The steady-state sideslip

From the rear slip angle, β = α_r + b r/U_x. Substituting α_r from the force balance:

$$\beta_{ss} = b\,\kappa - \frac{a\, m\, U_x^2\, \kappa}{L\, C_{\alpha r}}$$

At low speed the car's body points outward of the circle (β > 0 for a left turn: only kinematics). At high speed the rear tires' slip turns it inward, and β changes sign at U_x² = b L C_αr / (a m), about 13 m/s for this car.

11.3 The lookahead feedback

The feedback acts on the lateral error projected $x_{la} = 12$ m ahead of the CG along the car's heading: $e + x_{la}\cdot$(heading error). This combines lateral error and heading error with a single gain, and its effect is like a proportional–derivative controller on e, since the heading error approximately sets $\dot{e}$.

The heading error used is $e_\psi + \beta_{ss}$, not e_ψ. In steady cornering on the line the car's body is at an angle $-\beta_{ss}$ to the path (section 5.3). Without the correction, the feedback would fight that angle and pull the car off the line. With it, the feedback is zero exactly when the car is on the line at its steady-state attitude.

Kapania and Gerdes show this choice keeps the closed loop stable at the limit and the steady-state lateral error near zero, which pure pursuit's geometric law does not achieve.

11.4 Following a plan's steering (off by default)

With `plan_steering`, the feedforward is replaced by the NMPC plan's own steering angle, taken τ_{steer} ahead to allow for the actuator lag. The heading error becomes $e_\psi - e_{\psi,\text{plan}}$:

```
\delta_{\text{cmd}} = \delta_{\text{plan}} - k_p\big(e + x_{la}(e_\psi - e_{\psi,\text{plan}})\big)
```

It is off because it spun the car in 14 of the 240 corner outages and left the road in 2, mostly at the corner exit. There the heading feedback against the plan's body heading steered out while the car was still yawing in.

12. Pure pursuit: speed control and the per-axle force cap

Pure pursuit's longitudinal force tracks a speed reference with a feedforward of its slope, a proportional correction and drag compensation. The result is then capped twice: by what the car can deliver (`fx_limits`, section 4.5), and by the grip that cornering leaves on each axle.

12.1 The speed reference on its own profile

When it is not following a plan, pure pursuit uses a grip-limited speed profile (`speed_profile.cpp`) with an acceleration budget of 40% of the grip:

```
a = \text{grip\_use}\cdot\mu g = 0.4 \times 0.9 \times 9.81 = 3.53\ \text{m/s}^2
```

Cornering limit. At each point, 0.5 m apart, the speed may not exceed what the tightest of the home and passing lines allows at that budget: $v_c = \min(v_{max}, \sqrt{a/|\kappa_{line}|})$, since $a_y = v^2\kappa$.

Braking and accelerating share the budget. By the friction circle, cornering at speed v on curvature κ leaves this much along the path:

```
a_{\text{spare}}(v, \kappa) = \sqrt{\max\big(a^2 - (v^2\kappa)^2,\ 0\big)}
```

A backward pass makes every point slow enough to brake to the next one, from $v^2 = v_{next}^2 + 2 \cdot a \cdot \Delta s$. A forward pass makes it no faster than accelerating from the previous point allows:

```
v_i \leftarrow \min\Big(v_i,\ \sqrt{v_{i+1}^2 + 2\min\big(a_{\text{spare}}
(v_{i+1}, \kappa_{i+1}),\ a_{\text{brake}}\big)\,\Delta s}\Big), \qquad v_i
\leftarrow \min\Big(v_i,\ \sqrt{v_{i-1}^2 + 2\min\big(a_{\text{spare}}(v_{i-
1}, \kappa_{i-1}),\ a_{\text{accel}}\big)\,\Delta s}\Big)
```

Here $a_{brake} = 0.85 \times 7568 / 1845 = 3.49$ m/s² and $a_{accel} = 3$ m/s². On a closed circuit both passes run twice round the lap so they carry across the start line. With the road known only 40 m ahead, the backward pass runs every call over the known 40 m only, ending at the tightest corner's speed at 40% grip (`speed_within_sight`).

The resulting lap takes 98.5 s against the NMPC's 72.2 s. That gap is the main reason a takeover from the NMPC used to start with a large speed error (section 13).

12.2 The control law

Along the profile $v(s)$, the acceleration needed is $dv/dt = (dv/ds)(ds/dt) \approx v \cdot dv/ds$. A proportional term with gain $k_{speed} = 3$ s⁻¹ corrects the speed error, and drag (section 4.2) is added back:

```
a_{\text{cmd}} = v_{\text{ref}}\,\frac{dv_{\text{ref}}}{ds} +
k_{\text{speed}}\,(v_{\text{ref}} - U_x), \qquad F_x = m\, a_{\text{cmd}} +
F_{\text{drag}}(U_x)
```

A 1 m/s error asks for 3 m/s², about 5.5 kN. With the road range limited, a_{cmd} is also capped at a_{accel}.

12.3 The speed reference when following a plan

Following an NMPC plan, v_{ref} is the plan's U_x at the car's s, and its slope is taken from the plan 1 m further on. The NMPC plans at the limit, though, and tracking that within the force cap would leave no force to correct errors. So each plan speed is first capped at

what 90% of the grip holds the line at, with the line's curvature taken as the largest over the next 0.5 s:

```
v_{\text{ref}} = \min\left(v_{\text{plan}},\ \sqrt{\frac{0.9\,\mu g}
{\max_{\text{next } 0.5\,s}|\kappa_{\text{line}}|}}\right)
```

12.4 The per-axle force cap: derivation

The first crash under pure pursuit came from braking at the full 7568 N mid-hairpin. The car reached 7° of sideslip and spun. A whole-car friction circle did not prevent it: at 0.7 g it still allowed the full brake force. The problem is the rear axle.

One axle's condition. An axle carries a share φ of F_x and a lateral force F_y, and its load changes with F_x through the load transfer (section 3.2). Within a fraction η = `axle_grip_use` = 0.9 of its friction:

```
(\phi F)^2 + F_y^2 \le \big(\eta\mu\, (W + \sigma h F)/L\big)^2
```

Here F = |F_x|, W L is the axle's static load times L (m g b at the front, m g a at the rear), and σ = +1 where the axle gains load (front when braking, rear when driving) and −1 where it loses it.

The lateral force each axle must carry. In steady state the axles share m·a_y as F_yf = m a_y b/L and F_yr = m a_y a/L (section 11.1). a_y is the larger of the measured |U_x r| and what the line asks, U_x²|κ_line|. If the tire model at the measured slip angle says an axle is carrying more (as a sliding rear does), that larger value is used.

Solving for F. Write F_0 = ημW/L and k = σημh/L. Expanding the condition gives a quadratic in F:

```
(\phi^2 - k^2)\,F^2 - 2F_0 k\, F + (F_y^2 - F_0^2) \le 0
\quad\Longleftrightarrow\quad A F^2 - 2BF + C \le 0
```

If the cornering alone uses the grip (F_y ≥ F_0), the cap is zero. Otherwise C < 0, so there is a real positive root, and the largest allowed F is:

```
F_{\max} = \frac{B + \sqrt{B^2 - AC}}{A}
```

The cap is the smaller of the two axles' F_max, for braking and for driving separately.

12.5 A worked example: braking at 0.7 g

At a_y = 0.7 g the rear must carry F_yr = 1845 × 6.87 × 1.168 / 3.004 = 4926 N. Full braking at 7568 N puts 3784 N on the rear and drops its load to 5903 N, where 0.9 × 5903 = 5313 N is available. The rear would need √(4926² + 3784²) = 6212 N, 117% of its grip, so it slides.

The cap for the rear, with $\varphi = 0.5$, $F_0 = 0.81 \times 7037 = 5700$ N and $k = -0.81 \times 0.45 / 3.004 = -0.121$, comes to 3.66 kN. The front's cap is 15.0 kN, so the rear sets the limit: pure pursuit may brake at no more than 3.66 kN in that corner, under half the brakes' force.

12.6 The final force

```
F_x = \operatorname{clamp}\Big(m\,a_{\text{cmd}} + F_{\text{drag}},\
\max\big(F_{x,\min},\ -F_{\max}^{\text{brake}}\big),\ \min\big(F_{x,\max},\
F_{\max}^{\text{drive}}\big)\Big)
```

13. Fallback logic

An input is commanded every 20 ms whatever the NMPC does. The command thread blends the two controllers' inputs with a weight that ramps over 0.2 s, and pure pursuit meanwhile follows the NMPC's last plan, so a takeover carries on what the NMPC was doing (`fallback_control.cpp`).

13.1 When the NMPC is wanted

At each command the thread checks the plan in use. The NMPC's input is wanted only if all three hold:

- **In plan:** the current time falls inside the plan's horizon, so there is a planned input for it.
- **Not late:** no solve is under way whose plan should already have taken over.
- **No failure:** the latest solve converged, and the NMPC has not dropped out.

Situation	NMPC wanted	What the NMPC's part of a blend is
plan in use, next solve on time	yes	the plan's input for this moment
solve past its 200 ms period	no (late)	the last plan carried on past its period
solve did not converge	no (failure)	the last plan carried on
last plan's 4 s horizon has run out	no	the last input planned
NMPC dropped out (`drop_nmpc` , scenarios)	no	the last plan, then its last input

A late solve is kept, not dropped. When its plan arrives it takes over from the input it planned for that moment, and the weight ramps back.

13.2 The blend

The weight w is pure pursuit's share of the command. Each command moves it by at most one step towards its target:

```
w \leftarrow w + \operatorname{clamp}\big(w^\star - w,\ -\Delta,\
\Delta\big), \qquad w^\star = \begin{cases} 0 & \text{NMPC wanted} \\ 1 &
\text{otherwise} \end{cases}, \qquad \Delta = \frac{1}{f_{\text{cmd}}}\cdot
T_{\text{blend}}} = \frac{1}{50 \times 0.2} = 0.1
```

```
u = (1 - w)\, u_{\text{NMPC}} + w\, u_{\text{PP}}
```

So a full handover either way takes 10 commands, 0.2 s. Both inputs, steering rate and F_x, are blended linearly. Until the NMPC has produced any input, w = 1. Until pure pursuit has, w = 0.

Why a ramp. Switching instantly would step both inputs. A jump in F_x shifts the load between the axles at once, and a jump in steering rate jerks the steering. Either can unsettle a car that is near the limit in a corner, which is exactly when a handover is most likely.

13.3 Pure pursuit following the plan

Each time a new plan comes into use, the pure pursuit thread passes it to `PurePursuit::follow` as a reference. The reference is the plan's 49 samples of s, e_y, U_x, e_ψ and δ, valid until the plan's horizon ends (t_0 + 4 s). Samples are kept only while s increases.

Wherever the car's s lies within the reference, and until it expires, pure pursuit:

- steers onto the plan's e_y (interpolated at the car's s) as its target line, using the law of section 11;
- tracks the plan's U_x, capped as in section 12.3;
- sets its own target line to the plan's e_y, so the line slides on from there once the reference ends.

Beyond the plan it returns to its own lane and profile.

Measured effect. With solves forced late (horizon 4 s, period 0.075 s, 206 of 953 solves late), sampled over a lap at 25 Hz:

Measure	Following the plan	Own lane and profile
F_x gap to the NMPC, rms	1738 N	8020 N
Time more than 2 kN apart	18%	94%

Measure	Following the plan	Own lane and profile
Samples with pure pursuit below −6 kN	21	1432
Steering-rate gap, rms	23.6 deg/s	30.3 deg/s

13.4 Planning from where the car will be

The NMPC thread starts a solve at plant time t for a plan beginning at t_0 = t + T. It predicts the state at t_0 by integrating the bicycle model with RK4 in 5 ms steps. The inputs are those the plan in use (or the one about to take over) will send, if the NMPC alone is in command. Otherwise it holds the input applied now, since the blend's future share is not known.

The solve is then given the time t_0 and the predicted state. So the plan's first input is meant for exactly the moment it starts to be played.

14. Results, limitations and open work

The NMPC laps the circuit 25 s faster than pure pursuit on both offline plants. With the new laws, the fallback carries the car through 117 of 120 right-turn outages and 78 of 120 U-turn outages without leaving the road. It runs wide in the rest. CARLA itself has not yet been run with these settings.

14.1 Laps (README)

Plant	Controller	Lap	Speed	Passed	Asphalt margin	Peak sideslip
bicycle	NMPC	72.7 s	41–69 km/h	2/6	0.20 m	7.0°
bicycle	pure pursuit	98.2 s	24–61 km/h	1/6	0.37 m	5.4°
Chrono	NMPC	70.4 s	49–72 km/h	2/6	0.21 m	7.1°
Chrono	pure pursuit	97.9 s	24–61 km/h	1/6	0.31 m	4.2°
Chrono, exact sensing	NMPC	66.4 s	50–80 km/h	3/6	0.11 m	7.1°

With `nmpc_fallback` (4 s horizon, 200 ms period) on the bicycle stand-in: 73.6 s, 0.09 m asphalt margin, all 368 solves on time.

14.2 Corner outages (`results/scenarios/summary.md`)

15 radii from 15 to 85 m. The NMPC drops out at one of 8 points through a right turn or a U-turn, giving 120 runs per turn and fallback.

Turn	Fallback	Clean	Body over the edge	On the grass, off the road or spun	Worst margin	Peak pure pursuit brake (mean)
right	pure pursuit as it was	4	13	103	−4.00 m	7568 N
right	following the plan, old laws	29	58	33	−4.01 m	3438 N
right	**new laws, following the plan (default)**	**117**	**3**	**0**	**−0.23 m**	2770 N
right	new laws, plan's steering	86	22	12	−3.51 m	2655 N
right	new laws, own lane and profile	103	17	0	−0.74 m	2744 N
U-turn	pure pursuit as it was	5	13	102	−4.10 m	7568 N
U-turn	following the plan, old laws	26	21	73	−4.01 m	4002 N
U-turn	**new laws, following the plan (default)**	**78**	**42**	**0**	**−1.25 m**	3875 N
U-turn	new laws, plan's steering	84	29	7	−3.40 m	3625 N
U-turn	new laws, own lane and profile	110	10	0	−0.82 m	5507 N

"Worst margin" is the body's distance inside the asphalt at worst; negative means over the edge.

14.3 Known limitations

- **Model mismatch.** The NMPC holds C_α at the static load and has no tire lag or steering lag (section 2). It relies on feedback, through replanning, to absorb these.
- **Open-loop playback.** Each plan is played for 200 ms without feedback. In CARLA this roughly triples the steering swing on the straights compared with offline (2° against 0.6°).
- **Curvature from the previous plan.** κ and the speed limit are taken at the previous plan's positions, not the current solution's. They are exact only once successive plans agree.
- **Lumped axles.** No track width, so no yaw from a left–right force difference, and no roll dynamics.
- **Friction-circle derating, not combined slip.** Longitudinal slip is not modelled; F_x is taken as delivered.

- **Obstacle prediction.** Other cars are assumed to hold their speed and lateral offset over 4 s.
- **Pure pursuit's own profile** uses 40% of the grip, so a takeover beyond the plan's horizon starts with a large speed error.

14.4 Open work (`docs/todo.md`)

- [ ] Run a lap or two of the 4 s `nmpc_fallback` setup on the CARLA server.
- [ ] Long U-turns: the 4 s plan runs out mid-corner, pure pursuit drops to its 40%-grip profile, wants to brake hard at 0.8 g where the cap allows almost nothing, and runs wide (most of the 42 edge runs). Next step: a speed target that carries on from the plan's.
- [ ] Try a long outage, several periods without a plan.
- [ ] Give pure pursuit the same speed targets as the NMPC once the plan has run out.
- [ ] Add yaw-rate feedback to pure pursuit's steering, standing in for the NMPC's sideslip limit.
- [ ] Smoother pure pursuit steering: it runs at the rate limit most of the time (τ_steer = 0.05 s).